



LUIZ OTÁVIO ANDRADE SOARES

**GO X JAVA E GRPC X REST: UM ESTUDO
EMPÍRICO**

LAVRAS – MG

2024

LUIZ OTÁVIO ANDRADE SOARES

GO X JAVA E GRPC X REST: UM ESTUDO EMPÍRICO

Artigo apresentado à Universidade Federal de Lavras, como parte das exigências do Curso de Ciência da Computação, para a obtenção do título de Bacharel.

Prof. DSc. Ricardo Terra Nunes Bueno Vilela
Orientador

LAVRAS – MG

2024

**Ficha catalográfica elaborada pela Coordenadoria de Processos Técnicos
da Biblioteca Universitária da UFLA**

Soares, Luiz Otávio Andrade

Go x Java e gRPC x REST: Um estudo empírico / Luiz Otávio
Andrade Soares. 2^a ed. rev., atual. e ampl. – Lavras : UFLA, 2024.
53 p. :

Monografia(graduação)–Universidade Federal de Lavras, 2024.
Orientador: Prof. DSc. Ricardo Terra Nunes Bueno Vilela.
Bibliografia.

1. TCC. 2. Monografia. 3. Dissertação. 4. Tese. 5. Trabalho
Científico – Normas. I. Universidade Federal de Lavras. II. Título.

CDD-808.066

LUIZ OTÁVIO ANDRADE SOARES

**GO X JAVA E GRPC X REST: UM ESTUDO EMPÍRICO
GO VS. JAVA AND GRPC VS. REST: AN EMPIRICAL STUDY**

Artigo apresentado à Universidade Federal de Lavras, como parte das exigências do Curso de Ciência da Computação, para a obtenção do título de Bacharel.

APROVADA em 19 de Julho de 2024.

Prof. DSc. Ricardo Terra Nunes Bueno Vilela DCC - UFLA
Prof. DSc. Rafael Serapilha Durelli DCC - UFLA
Prof. DSc. Mayron César de Oliveira Moreira DCC - UFLA

Prof. DSc. Ricardo Terra Nunes Bueno Vilela
Orientador

**LAVRAS – MG
2024**

À minha família, amigos e ao meu amor

AGRADECIMENTOS

Primeiramente, agradeço à minha família, cujo apoio incondicional foi fundamental durante toda esta jornada. Um agradecimento especial à minha avó Maria Alice, que não apenas me inspirou com sua sabedoria e carinho, mas também viabilizou a minha oportunidade de estudar fora. Sem o seu apoio, essa conquista não seria possível. Sua generosidade e dedicação são fontes de força e motivação que levarei comigo para sempre.

À minha mãe Eva, que sempre acreditou no meu potencial e me incentivou a seguir em frente, mesmo nos momentos mais desafiadores, meu profundo agradecimento. Ao meu irmão João Bruno, que, com sua amizade e carinho, tornou cada etapa deste caminho mais leve e significativa.

Gostaria de agradecer também à minha república, Último Gole, por todos os momentos de descontração e companheirismo, que foram essenciais para manter o equilíbrio durante os estudos. A convivência com vocês foi uma parte importante da minha experiência universitária, e sou grato por cada instante compartilhado.

Um agradecimento especial à minha namorada, Laura Sarto, pelo apoio, paciência e por estar ao meu lado em todos os momentos, celebrando as conquistas e oferecendo conforto nas dificuldades. Sua presença foi um alicerce sólido ao longo desta jornada.

Por fim, agradeço a todos que, de alguma forma, contribuíram para a realização deste trabalho. Agradeço a todos os professores, amigos e colegas que fizeram parte dessa trajetória.

*No meio do caminho tinha uma pedra, tinha uma pedra no meio do caminho.
(Carlos Drummond de Andrade)*

RESUMO

Este trabalho investiga a influência das linguagens de programação Java e Go e das arquiteturas de comunicação REST e gRPC no desempenho de APIs. A questão de pesquisa #1 investigou qual par (*linguagem, arquitetura-comunicação*) provê melhor desempenho. Por um lado, pares (Go, gRPC) e (Java, gRPC) se destacaram em requisições de tamanhos usuais (*StdSize*) possivelmente devido à eficiência na compactação de dados e na redução de latência oferecida pelo gRPC. Por outro lado, pares (Go, REST) e (Java, REST) se destacaram em requisições de grande volume (*LargeSize*) provavelmente devido à flexibilidade do REST em lidar com grandes volumes de dados sem estrutura rígida. A questão de pesquisa #2, de forma complementar, investigou a influência de cada fator no desempenho. Um projeto fatorial 2^k concluiu que enquanto a linguagem de programação exerce uma influência de 2%, a arquitetura de comunicação influencia 94%.

Palavras-chave: Linguagens de programação, estilo arquitetural de comunicação, estudo empírico

ABSTRACT

This study investigates the influence of the Java and Go programming languages and the REST and gRPC communication architectures on the performance of APIs. Research Question #1 examined which pair (language, communication architecture) provides better performance. On one hand, the pairs (Go, gRPC) and (Java, gRPC) excelled in standard-size requests (StdSize), possibly due to the efficiency in data compression and latency reduction offered by gRPC. On the other hand, the pairs (Go, REST) and (Java, REST) stood out in handling large-volume requests (LargeSize), likely due to REST's flexibility in managing large data volumes without a rigid structure. Complementarily, Research Question #2 investigated the influence of each factor on performance. A 2kr factorial design concluded that while the programming language exerts a 2% influence, the communication architecture influences 94%.

Keywords: Programming Languages, Communication Architectural Style, Empirical Study

LISTA DE FIGURAS

Figura 2.1 – Metodologia	31
Figura 2.2 – Desempenho em requisições <i>StdSize</i>	35
Figura 2.3 – Desempenho em requisições <i>LargeSize</i>	35

LISTA DE TABELAS

Tabela 2.1 – Mediana do desempenho com desvio padrão	36
Tabela 2.2 – Wilcoxon <i>Rank Sum</i> : resultados <i>p</i> -valor	38
Tabela 2.3 – Kruskal-Wallis: resultados	38
Tabela 2.4 – <i>Rankings</i> das implementações	39
Tabela 2.5 – Projeto Fatorial 2^2r - <i>StdSize</i>	41
Tabela 2.6 – Influência dos Fatores no Desempenho	42

SUMÁRIO

1	INTRODUÇÃO	21
2	ARTIGO 1 - Go x Java e gRPC x REST: Um estudo empírico	23
2.1	Introdução	23
2.2	<i>Background</i>	25
2.2.1	Linguagens de Programação	25
2.2.2	Estilos arquiteturais de comunicação	26
2.2.3	Procedimentos Estatísticos	27
2.3	Estudo Empírico	28
2.3.1	Questões de Pesquisa	29
2.3.2	Metodologia	29
2.3.3	Ambiente	31
2.3.4	Aplicação Alvo	32
2.3.5	Coleta de Dados	33
2.3.6	Análise de Desempenho (<i>QP#1</i>)	34
2.3.7	Influência dos Fatores (<i>QP#2</i>)	40
2.3.8	Ameaças à validade	43
2.3.9	Replicabilidade	44
2.4	Trabalhos Relacionados	44
2.5	Conclusão	47
3	CONCLUSÃO	49
	REFERÊNCIAS	51

1 INTRODUÇÃO

As APIs (*Application Programming Interfaces*) desempenham um papel crucial no desenvolvimento de sistemas, facilitando a integração e comunicação entre diferentes sistemas de software e plataformas, permitindo assim uma maior modularidade, escalabilidade e reutilização de código (KUMAR et al., 2023). Na prática usual adotada por fábricas de software, utiliza-se o protocolo HTTPS (*Hypertext Transfer Protocol Secure*) para facilitar a comunicação e transferência de dados na Internet e o estilo arquitetural de comunicação REST (*Representational State Transfer*) para organizar e acessar serviços web de maneira padronizada (FIELDING et al., 1999; FIELDING, 2000). Isso se consolidou como o padrão usual para a criação de sistemas distribuídos robustos (BOLANOWSKI et al., 2022; KUMAR et al., 2023; LEE; LIU, 2022).

A ascensão de Go e gRPC, promovida significativamente pelo Google, marca uma evolução nas tecnologias de desenvolvimento de software (DONOVAN; KERNIGHAN, 2012; GOOGLE, 2015). Go destaca-se pela sua simplicidade e eficiência no manejo de operações concorrentes, enquanto gRPC revitaliza o conceito de chamadas de procedimento remoto (RPC) ao integrar funcionalidades do HTTP/2, facilitando comunicações rápidas e eficientes por meio de *Protocol Buffers* para serialização de dados. Essas características são particularmente benéficas para microserviços e aplicações que demandam alto desempenho e baixa latência.

Este artigo, portanto, conduz um estudo empírico que emprega procedimentos estatísticos rigorosos para quantificar as melhorias oferecidas por uma linguagem de programação e um estilo arquitetural de comunicação considerados promissores. Partindo da presunção de que Java é uma linguagem consolidada no mercado, assim como REST é quase sempre adotado como o estilo arquitetural de comunicação, este estudo (i) investiga a combinação mais eficiente entre as linguagens de programação Go e Java e os estilos arquiteturais de comunicação

gRPC e REST e (ii) quantifica a influência da linguagem de programação e do estilo arquitetural de comunicação na eficiência.

A partir de quatro implementações – (Go, gRPC), (Go, REST), (Java, gRPC) e (Java, REST) – de uma mesma aplicação, a questão de pesquisa #1 investigou qual par (*linguagem, arquitetura-comunicação*) provê melhor desempenho e comprovou diferença estatística entre todos os pares. Além disso, as análises indicaram que os pares (Go, gRPC) e (Java, gRPC) se destacam em requisições de tamanhos usuais (*StdSize*) possivelmente devido à eficiência na compactação de dados e na redução de latência oferecida pelo gRPC. Por outro lado, pares (Go, REST) e (Java, REST) se destacaram em requisições de grande volume (*LargeSize*) provavelmente devido à flexibilidade do REST em lidar com grandes volumes de dados sem estrutura rígida. A questão de pesquisa #2, de forma complementar, investigou a influência de cada fator no desempenho por meio de um projeto fatorial 2^k . Essa análise concluiu que a linguagem de programação exerce uma influência de 2%, enquanto que a arquitetura de comunicação influencia 94%.

Este documento está organizado como a seguir. A Seção 2 contém o artigo completo, enquanto a seção 3 contém considerações finais acerca do estudo conduzido.

2 ARTIGO 1 - GO X JAVA E GRPC X REST: UM ESTUDO EMPÍRICO

2.1 Introdução

As APIs (*Application Programming Interfaces*) desempenham um papel crucial no desenvolvimento de sistemas, facilitando a integração e comunicação entre diferentes sistemas de software e plataformas, permitindo assim uma maior modularidade, escalabilidade e reutilização de código (KUMAR et al., 2023). Na prática usual adotada por fábricas de software, utiliza-se o protocolo HTTPS (*Hypertext Transfer Protocol Secure*) para facilitar a comunicação e transferência de dados na Internet e o estilo arquitetural de comunicação REST (*Representational State Transfer*) para organizar e acessar serviços web de maneira padronizada (FIELDING et al., 1999; FIELDING, 2000). Isso se consolidou como o padrão usual para a criação de sistemas distribuídos robustos (BOLANOWSKI et al., 2022; KUMAR et al., 2023; LEE; LIU, 2022).

A ascensão de Go e gRPC, promovida significativamente pelo Google, marca uma evolução nas tecnologias de desenvolvimento de software (DONOVAN; KERNIGHAN, 2012; GOOGLE, 2015). Go destaca-se pela sua simplicidade e eficiência no manejo de operações concorrentes, enquanto gRPC revitaliza o conceito de chamadas de procedimento remoto (RPC) ao integrar funcionalidades do HTTP/2, facilitando comunicações rápidas e eficientes por meio de *Protocol Buffers* para serialização de dados. Essas características são particularmente benéficas para microsserviços e aplicações que demandam alto desempenho e baixa latência.

Na literatura atual, é comum encontrar estudos comparativos de tecnologias que frequentemente apresentam metodologias pouco definidas (ABHINAV et al., 2020; KAMIŃSKI et al., 2023; CHAVAN; KADAM; MISTRY, 2023). Além disso, nota-se a falta de rigor na aplicação de métodos estatísticos para a análise dos resultados, ou até ausência de métodos estatísticos (ŚLIWA; PAŃCZYK, 2021;

KAMIŃSKI et al., 2023; BERG; REDI, 2023). A maioria desses estudos se baseia em experimentos práticos sem uma estrutura metodológica robusta, o que pode limitar a validade e a confiabilidade das conclusões. Em contraste, este trabalho se diferencia ao empregar uma metodologia bem definida e o uso de métodos estatísticos rigorosos para corroborar as análises.

Este artigo, portanto, conduz um estudo empírico que emprega procedimentos estatísticos rigorosos para quantificar as melhorias oferecidas por uma linguagem de programação e um estilo arquitetural de comunicação considerados promissores. Partindo da presunção de que Java é uma linguagem consolidada no mercado, assim como REST é quase sempre adotado como o estilo arquitetural de comunicação, este estudo (i) investiga a combinação mais eficiente entre as linguagens de programação Go e Java e os estilos arquiteturais de comunicação gRPC e REST e (ii) quantifica a influência da linguagem de programação e do estilo arquitetural de comunicação na eficiência.

A partir de quatro implementações – (Go, gRPC), (Go, REST), (Java, gRPC) e (Java, REST) – de uma mesma aplicação, a questão de pesquisa #1 investigou qual par (*linguagem, arquitetura-comunicação*) provê melhor desempenho e comprovou diferença estatística entre todos os pares. Além disso, as análises indicaram que os pares (Go, gRPC) e (Java, gRPC) se destacam em requisições de tamanhos usuais (*StdSize*) possivelmente devido à eficiência na compactação de dados e na redução de latência oferecida pelo gRPC. Por outro lado, pares (Go, REST) e (Java, REST) se destacaram em requisições de grande volume (*LargeSize*) provavelmente devido à flexibilidade do REST em lidar com grandes volumes de dados sem estrutura rígida. A questão de pesquisa #2, de forma complementar, investigou a influência de cada fator no desempenho por meio de um projeto fatorial 2^k . Essa análise concluiu que a linguagem de programação exerce uma influência de 2%, enquanto que a arquitetura de comunicação influencia 94%.

Este artigo está organizado como a seguir. A Seção 2.2 provê uma visão geral das linguagens de programação, dos estilos arquiteturais de comunicação e dos métodos estatísticos utilizados. A Seção 2.3 descreve o estudo empírico incluindo questões de pesquisa, metodologia, coleta de dados, análise de desempenho e influência dos fatores. A Seção 2.4 apresenta trabalhos relacionados e a Seção 2.5 conclui e enumera trabalhos futuros.

2.2 Background

Esta seção introduz conceitos fundamentais ao artigo.

2.2.1 Linguagens de Programação

Go, também conhecida como *Golang*, é uma linguagem de programação lançada publicamente pelo Google em 2009. Projetada por Robert Griesemer, Rob Pike e Ken Thompson, Go visa simplificar o desenvolvimento de software com suporte robusto para programação concorrente e gerenciamento eficiente de memória. A linguagem oferece uma sintaxe limpa e recursos como *goroutines* e *channels*, que facilitam a construção de sistemas distribuídos escaláveis. A capacidade de Go de compilar rapidamente e sua eficiência em tempo de execução a tornam ideal para a criação de serviços de alto desempenho (DONOVAN; KERNIGHAN, 2012) o que justifica a sua ampla adoção da indústria (COX et al., 2022).

Java é uma linguagem de programação de propósito geral que é amplamente utilizada para desenvolvimento de software (TIOBE Software, 2024). Criada por James Gosling e lançada pela *Sun Microsystems* em 1995, Java é conhecida por seu princípio de *Write Once, Run Anywhere* (WORA), que permite que o código Java seja executado em qualquer dispositivo que suporte a *Java Virtual Machine* (JVM) (SINGHAL; NGUYEN, 1998). De acordo com o índice TIOBE, que mede a popularidade das linguagens de programação com base em diversos fa-

tores, Java frequentemente aparece entre as linguagens mais utilizadas, destacando sua relevância contínua na indústria (TIOBE Software, 2024).

2.2.2 Estilos arquiteturais de comunicação

O HTTP é o protocolo subjacente usado pela *World Wide Web* para a troca de informações entre navegadores e servidores. O HTTP é um protocolo de aplicação que segue um modelo de requisição-resposta. Um cliente envia uma requisição HTTP para um servidor, que então responde com uma mensagem de *status* e os dados solicitados. Com a evolução do uso da web, surgiu o HTTPS, que adiciona uma camada de segurança ao HTTP, utilizando os protocolos SSL/TLS para criptografar as comunicações entre cliente e servidor, o que é essencial para a proteção dos dados sensíveis (ASRATIAN, 2023). Além disso, a simplicidade do HTTP permitiu sua adoção generalizada, e a introdução de HTTP/2 trouxe melhorias significativas em termos de desempenho, incluindo multiplexação de requisições e compressão de cabeçalhos (FIELDING et al., 1999; BELSHE; PEON; THOMSON, 2015).

A arquitetura REST é um estilo de arquitetura de comunicação para sistemas distribuídos, como a *World Wide Web* (FIELDING, 2000). Proposta por Roy Fielding em sua tese de doutorado em 2000, REST define um conjunto de restrições que resultam em um sistema escalável, de fácil manutenção e eficiente. As APIs construídas sob a arquitetura REST utilizam métodos HTTP (como GET, POST, PUT e DELETE) para manipular recursos, que são identificados por URIs (*Uniform Resource Identifiers*). Um dos aspectos chave do REST é o uso de JSON (*JavaScript Object Notation*) como formato padrão para a troca de dados entre clientes e servidores (CROCKFORD, 2006).

É importante diferenciar REST de RESTful, onde RESTful refere-se à implementação das restrições REST de maneira completa e fiel, enquanto REST pode ser utilizado de forma mais flexível, não aderindo estritamente a todas as restrições

originais de REST. Pelo foco experimental, este trabalho adota o estilo REST que permite uma abordagem mais prática e adaptável e mantém os princípios fundamentais da arquitetura.

A simplicidade e a padronização de REST têm contribuído para sua ampla adoção em diversas aplicações web (BOLANOWSKI et al., 2022; KUMAR et al., 2023; LEE; LIU, 2022).

RPC (*Remote Procedure Call*) é um estilo arquitetural de comunicação que permite a um programa executar procedimentos em outro espaço de endereço, geralmente em outro computador na rede, como se fossem chamadas locais (BIRRELL; NELSON, 1984). De forma sucinta, RPC abstrai os detalhes da rede, proporcionando uma interface mais simples para os desenvolvedores. O gRPC é uma evolução moderna do RPC desenvolvida pelo Google. Utilizando HTTP/2 como protocolo de transporte, gRPC oferece vantagens como suporte a comunicação bidirecional, multiplexação de requisições e maior eficiência na transmissão de dados (GOOGLE, 2015).

2.2.3 Procedimentos Estatísticos

Este trabalho emprega uma combinação de testes estatísticos não paramétricos e métodos de análise de variância para explorar as influências significativas e interações entre as variáveis estudadas.

Teste de Shapiro-Wilk. É um método estatístico usado para verificar se uma amostra provém de uma distribuição normal. Esse teste compara a ordem dos dados da amostra com uma distribuição normal esperada, utilizando uma estatística de teste W . Se W está próximo de 1, sugere-se que a amostra pode ser considerada normalmente distribuída. Essencial em pesquisas que dependem de análises paramétricas, o teste de Shapiro-Wilk é particularmente eficaz para amostras pequenas, embora sua sensibilidade possa ser uma limitação para amostras grandes, onde pequenos desvios de normalidade podem levar à rejeição da hipótese nula (MIOT, 2017).

Teste de Wilcoxon. É um teste não paramétrico utilizado para comparar duas amostras emparelhadas, a fim de determinar se suas distribuições populacionais diferem significativamente. Para amostras não pareadas utiliza-se a versão *Rank-Sum* do teste (DATTA; SATTEN, 2005). É particularmente útil quando a distribuição dos dados não pode ser assumida como normal, uma condição frequentemente encontrada em dados de desempenho de software. Esse teste foi aplicado para analisar as diferenças de desempenho entre as implementações de REST e gRPC (POKROPP, 1992).

Teste de Kruskal-Wallis. É um teste empregado quando se deseja comparar três ou mais grupos independentes sem assumir uma distribuição normal dos dados. Esse teste é essencial para avaliar a variação no desempenho entre diferentes configurações linguísticas e arquitetônicas, oferecendo uma análise robusta que complementa as comparações bivariadas do teste de Wilcoxon. Os resultados obtidos fornecem *insights* sobre quais configurações oferecem melhor desempenho em termos de eficiência e velocidade (HABIBZADEH, 2024).

Projeto fatorial $2^k r$. É um projeto conduzido para a análise de dois ou mais fatores de forma simultânea e seus efeitos interativos sobre a variável resposta. No contexto deste estudo, as variáveis analisadas incluem as linguagens de programação Go e Java, e as arquiteturas de comunicação gRPC e REST, com repetições (r) para garantir a precisão dos resultados. Ele não apenas identifica as configurações mais eficientes, como também revela como essas escolhas interagem entre si para influenciar o desempenho geral (JAIN, 1991).

2.3 Estudo Empírico

Esta seção descreve o estudo empírico realizado para investigar o desempenho de aplicações utilizando as linguagens de programação Go e Java em conjunto com os estilos arquiteturais de comunicação gRPC e REST.

2.3.1 Questões de Pesquisa

1. **Qual é a combinação mais eficiente entre as linguagens de programação Go e Java e os estilos arquiteturais de comunicação gRPC e REST para otimizar o desempenho de aplicações?**

Essa questão busca determinar qual par de linguagem e arquitetura de comunicação – (Go, gRPC), (Go, REST), (Java, gRPC) e (Java, REST) – oferece a melhor eficiência em termos de desempenho, em diferentes cenários de aplicação.

2. **Quanto cada tecnologia, seja linguagem (Go e Java) ou estilo arquitetural de comunicação (gRPC e REST) influencia no tempo de execução das aplicações?**

O objetivo dessa questão é mensurar o quanto as escolhas de linguagem de programação e estilo arquitetural de comunicação impactam no desempenho das aplicações.

Essas questões são fundamentais para compreender as nuances de desempenho, o que pode ajudar desenvolvedores a fazer escolhas corretas para seus projetos.

2.3.2 Metodologia

Conforme ilustrado na Figura 2.1, o procedimento metodológico adotado pode ser descrito em cinco etapas:

1. Desenvolvimento de quatro implementações de uma mesma aplicação que referem-se às combinações das linguagens de programação e estilos arquiteturais de comunicação empregados neste estudo.

Resultado esperado: implementações (Go, gRPC), (Go, REST), (Java, gRPC) e (Java, REST).

2. Condução de um experimento preliminar para estimar o tamanho de amostra ideal.

Resultado esperado: obtenção do n ideal.

3. Execução n vezes de cada uma das implementações com requisições de tamanho usual (*StdSize*) e de grande volume (*LargeSize*). Cada execução é repetida r vezes.

Resultado esperado: oito arquivos – quatro implementações $\times$ dois tamanhos de requisição – com n tempos de execução. Cada arquivo contendo r conjuntos de observações.

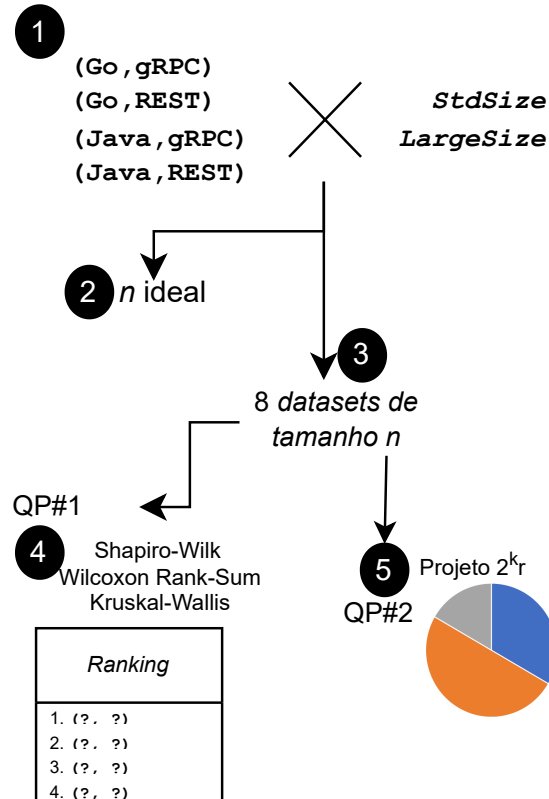
4. Aplicação do teste de Shapiro-Wilk para verificar a normalidade das distribuições. Caso os dados não sejam normais, (i) aplicação do teste de Wilcoxon *Rank-Sum* para verificar se os tempos de execução de cada uma das implementações são estatisticamente diferentes dentro do mesmo tamanho de requisição e (ii) aplicação do teste de Kruskal-Wallis para avaliar a variação no desempenho entre diferentes implementações, o que complementa as comparações bivariadas do teste de Wilcoxon.

Resultado esperado: ranking das implementações com validade estatística.

5. Condução do projeto fatorial $2^k r$ para determinar a influência dos fatores e suas interações sobre o desempenho das implementações.

Resultado esperado: percentual de influência de cada fator.

Figura 2.1 – Metodologia



2.3.3 Ambiente

O ambiente experimental foi configurado de forma a promover o menor erro experimental possível:

Hardware. Intel Core i7 de 9ª geração e 16 GB de RAM.

Sistema operacional. Ubuntu Server com instalação mínima, garantindo que nenhuma outra aplicação interferisse nos testes, proporcionando um ambiente controlado e consistente.

Software. apenas Java 17 e Go 1.16. A escolha das versões foi motivada por sua estabilidade e suporte de longo prazo, garantindo um ambiente de desenvolvi-

mento confiável.

Servidor de aplicação. Para Java, utilizou-se o Tomcat 10.1.8 para a implementação REST e gRPC Netty 1.56.1 para gRPC. Go, por outro lado, já possui funcionalidades embutidas para servir HTTP no pacote `net/http`, utilizado em ambas as implementações.

Coleta. Durante a coleta, a aplicação alvo, seja qual for a implementação, foi configurada para rodar com a máxima prioridade (*nice level* de -20), enquanto a aplicação cliente foi configurada com um *nice level* de -19.

2.3.4 Aplicação Alvo

No âmbito desse experimento, tem-se quatro implementações distintas de uma mesma aplicação, cada uma representando uma combinação única de linguagem de programação e estilo arquitetural de comunicação: (Go, gRPC), (Go, REST), (Java, gRPC) e (Java, REST). O propósito central da aplicação é a implementação de um único *endpoint*, o qual tem a função crítica de receber um conjunto de números e, a partir desse conjunto, selecionar e exibir três elementos em posições aleatórias. Essa metodologia deliberadamente simplificada foi adotada com o objetivo de isolar e quantificar com precisão o impacto exercido tanto pela linguagem de programação quanto pelo estilo arquitetural de comunicação sobre o desempenho geral do sistema. Essa abordagem reduz significativamente a possibilidade de interferência por variáveis externas ao experimento.

Além disso, a operação selecionada para ser realizada pela aplicação foi cuidadosamente escolhida para assegurar que o *payload* completo seja efetivamente transferido para o serviço em questão, prevenindo assim qualquer forma de otimização prematura na leitura e processamento dos dados. Essa premissa é de vital importância para o estudo, uma vez que uma parcela substancial das discrepâncias observadas entre os diferentes estilos arquiteturais advém especificamente

da maneira como os dados são encapsulados e transportados por meio da rede. Em consequência, é imperativo avaliar com minúcia o impacto que a transformação desses dados, de sua forma bruta para um formato que seja manejável pela linguagem de programação, tem sobre o desempenho geral.

Uma aplicação é responsável por orquestrar os testes e coletar dados de desempenho. São feitas n requisições para cada um dos seguintes tipos de *payloads*:

O *payload* “*StdSize*” compreende um arranjo de 200 números inteiros com valores aleatórios entre 0 e 1.000, totalizando 0,763 KB. Esse tamanho é representativo do tamanho médio de requisições HTTP comumente observadas em aplicações web, servindo como um *benchmark* relevante para análises de desempenho (The Chromium Projects, 2012). O *payload* “*LargeSize*” é concebido para ser exatamente 1.024 vezes maior que o “*StdSize*”, refletindo uma carga significativamente mais pesada. Isso eleva o tamanho do *payload* “*LargeSize*” para 781,312 KB.

Essa escala ampliada é crucial para testar o desempenho das implementações sob condições extremas de carga, permitindo uma investigação sobre como diferentes tamanhos de *payload* afetam o desempenho nas aplicações distribuídas.

2.3.5 Coleta de Dados

O procedimento experimental é delineado em $r = 5$ repetições, cada uma contendo $n = 77$ requisições. Utilizou-se uma fórmula que permitiu definir o número de requisições ideal para a amostra com base em um conjunto de observações preliminares. Considerando um nível de confiança de 95% e 9 graus de liberdade (10 observações preliminares), a variável z tem o valor de 1,833 e representa o valor crítico da distribuição t de *Student* para esse nível de confiança e graus de liberdade especificados. O parâmetro r é definido como o erro tolerável, fixado em 5%. A fórmula geral para o cálculo do tamanho da amostra (sz) é dada por (JAIN,

1991):

$$sz = \left(\frac{100 \cdot z \cdot s}{r \cdot \bar{x}} \right)^2 \quad (2.1)$$

onde:

- s é o desvio padrão das medições prévias,
- $\bar{x}$ é a média das medições prévias,
- z é o valor t de *Student* para o nível de confiança e
- r é o erro percentual tolerável.

Durante cada repetição, a aplicação calcula o tempo despendido de cada requisição e as armazena em memória. Esse processo é repetido até que todas as repetições previstas sejam concluídas. Na fase final, esses dados são exportados para oito arquivos com n tempos de execução, um arquivo para cada uma das quatro implementações e para cada um dos dois tamanhos de requisição, e importados no R para análises estatísticas (R Core Team, 2019).

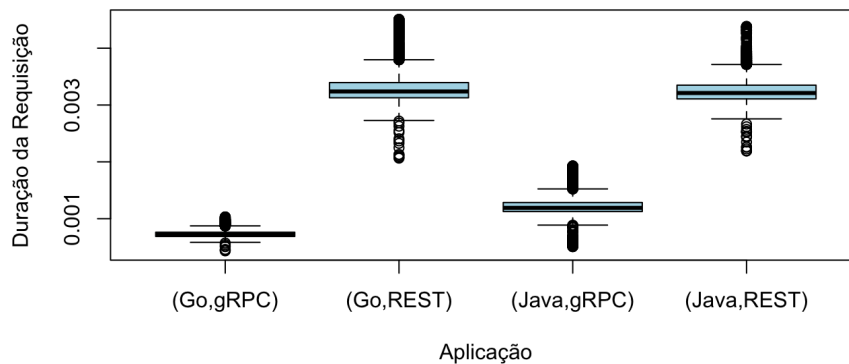
2.3.6 Análise de Desempenho (QP#1)

Esta seção provê respostas à questão de pesquisa #1 por meio da aplicação de procedimentos estatísticos em uma das cinco repetições dos dados coletados conforme descrito na Seção 2.3.5.

A Figura 2.2 ilustra um gráfico *boxplot* comparando os tempos de execução das quatro implementações (Go, gRPC), (Go, REST), (Java, gRPC) e (Java, REST) no cenário de requisições *StdSize*. Cada caixa representa a distribuição dos tempos de resposta de diferentes configurações de aplicativos. A parte inferior da caixa indica o primeiro quartil (Q1), ou seja, 25% dos dados são inferiores a esse valor, enquanto a parte superior mostra o terceiro quartil (Q3), com 75% dos dados abaixo desse ponto. O espaço entre Q1 e Q3, conhecido como intervalo interquartil (IQR), oferece uma medida da dispersão dos dados. A linha dentro da caixa

marca a mediana, que é o valor central dos dados. Pontos individuais, ou *outliers*, representam dados que se distanciam significativamente dos demais, geralmente definidos como valores que estão além de 1,5 vezes o IQR a partir dos quartis.

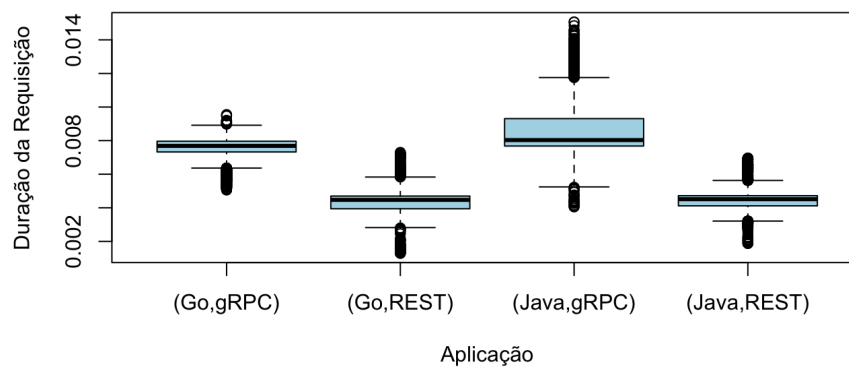
Figura 2.2 – Desempenho em requisições *StdSize*



No gráfico, é possível observar que combinações que utilizam gRPC têm desempenho melhor que as que utilizam REST, além de que a espessura das caixas indicam que existe uma variabilidade menor quando se utiliza a arquitetura de comunicação gRPC.

A Figura 2.3, de forma complementar, ilustra um gráfico *boxplot* similar, porém no cenário de requisições *LargeSize*. Nota-se no gráfico que aplicações que usam REST possuem desempenho superior as que usam gRPC.

Figura 2.3 – Desempenho em requisições *LargeSize*



A Tabela 2.1 reporta os dados de mediana e desvio padrão. Nota-se diferenças significativas entre os cenários de requisições *StdSize* e *LargeSize*. Por um lado, nas requisições *StdSize*, a implementação (Go, gRPC) apresentou o menor tempo médio de resposta, destacando-se também por seu baixo desvio padrão, o que indica uma regularidade notável no desempenho. Por outro lado, nas requisições *LargeSize*, a implementação (Go, REST) apresentou o menor tempo médio, complementado por um desvio padrão relativamente baixo, sugerindo relativa estabilidade mesmo sob condições de cargas de grande volume.

Tabela 2.1 – Mediana do desempenho com desvio padrão

<i>Implementação</i>	<i>StdSize</i>		<i>LargeSize</i>	
	<i>Mediana</i>	<i>Desvio</i>	<i>Mediana</i>	<i>Desvio</i>
(Go, gRPC)	0,00071	0,00006	0,00768	0,00070
(Go, REST)	0,00323	0,00028	0,00447	0,00067
(Java, gRPC)	0,00119	0,00017	0,00803	0,00134
(Java, REST)	0,00321	0,00023	0,00451	0,00066

Normalidade dos dados. A análise de normalidade dos dados desempenhou um papel crucial antes de proceder com testes estatísticos. Para os tempos de execução de todas as oito combinações, o teste de Shapiro-Wilk indicou que os dados *não* seguiam uma distribuição normal.

Como esse teste possui uma sensibilidade grande para pequenos desvios de normalidade (MIOT, 2017) quando utilizado em grandes quantidades de amostras (como neste estudo), métodos visuais de análise como gráficos Q-Q e histogramas foram empregados para confirmar a suposição e avaliar a distribuição dos tempos de resposta das implementações. Essas análises confirmaram a não-normalidade dos dados, um achado significativo que orientou a escolha de técnicas estatísticas adequadas para a análise subsequente (HABIBZADEH, 2024). A não-normalidade dos dados é uma característica comum em conjuntos de dados reais, especialmente em medições de desempenho de sistemas computacionais, onde fatores externos e a própria natureza das tecnologias podem influenciar grandemente a variação dos dados (HOLOVNIA et al., 2023).

Dada essa característica dos dados, optou-se por métodos não paramétricos para as análises seguintes. A escolha foi fundamentada pela capacidade desses métodos de fornecer resultados confiáveis sem a necessidade de pressupostos de normalidade, essenciais em testes paramétricos. Essa decisão metodológica assegura que as comparações de desempenho entre as diferentes combinações de linguagens e estilos arquiteturais de comunicação sejam válidas e não comprometidas por violações das premissas estatísticas (HOLOVNIA et al., 2023). Portanto, a análise de normalidade não apenas informou a escolha dos testes estatísticos apropriados, mas também reforçou a robustez das inferências estatísticas realizadas nesse estudo (HABIBZADEH, 2024).

Diferenças estatisticamente relevantes. Conforme abordado na Seção 2.2.3, o teste de Wilcoxon *Rank-Sum* é um método não paramétrico utilizado para comparar duas amostras independentes. O método é ideal para dados que não seguem uma distribuição normal e é aplicado quando se deseja determinar se duas distribuições possuem a mesma mediana sem assumir a normalidade dos dados. Dada a sua capacidade de adaptar-se às condições do estudo, o teste de Wilcoxon pode ser implementado em suas formas condicionais, com classificações naturais e intermediárias, para analisar eficazmente as diferenças entre as medianas (POKROPP, 1992).

A Tabela 2.2 reporta os resultados dos testes de Wilcoxon, tanto para requisições *StdSize* quanto para *LargeSize*. Os resultados indicaram diferenças estatisticamente significativas em todas as comparações entre as combinações. A comprovação de todos os pares serem estatisticamente diferentes permite comparar as medianas e classificar os melhores pares de tecnologias.

O teste de Kruskal-Wallis é especialmente útil para analisar variações em dados categóricos, como os avaliados nesse estudo, onde é analisado o impacto de diferentes linguagens de programação e estilos arquiteturais de comunicação no desempenho das aplicações. Esse teste foi empregado em conjunto com o teste de

Tabela 2.2 – Wilcoxon *Rank Sum*: resultados *p*-valor

Comparação	<i>StdSize</i>	<i>LargeSize</i>
(Java, REST) vs. (Java, gRPC)	< 2,2e-16	< 2,2e-16
(Java, REST) vs. (Go, REST)	4,532e-09	< 2,2e-16
(Java, REST) vs. (Go, gRPC)	< 2,2e-16	< 2,2e-16
(Java, gRPC) vs. (Go, REST)	< 2,2e-16	< 2,2e-16
(Java, gRPC) vs. (Go, gRPC)	< 2,2e-16	< 2,2e-16
(Go, REST) vs. (Go, gRPC)	< 2,2e-16	< 2,2e-16

Wilcoxon *Rank-Sum* para reforçar os resultados obtidos e garantir que as comparações entre as implementações sejam válidas.

Tabela 2.3 reporta os resultados do teste de Kruskal-Wallis. Primeiramente, conforme esperado, o teste indicou que tanto o estilo arquitetural de comunicação quanto a linguagem de programação têm um impacto significativo no desempenho das aplicações, com diferenças estatisticamente relevantes observadas tanto em requisições *StdSize* quanto *LargeSize*. Em segundo lugar, os resultados indicam que tanto a escolha do estilo arquitetural de comunicação quanto a linguagem de programação afetam o desempenho. Por último, mas não menos importante, quando as variáveis de estilo arquitetural de comunicação e linguagem de programação foram combinadas para análise, a interação entre esses dois fatores mostrou um efeito ainda mais pronunciado no desempenho. Estudos como os realizados por Habibzadeh (HABIBZADEH, 2024) ilustram a aplicabilidade do teste de Kruskal-Wallis em contextos diversos, reforçando a importância de considerar não apenas os efeitos isolados de cada variável, mas também como elas interagem entre si.

Tabela 2.3 – Kruskal-Wallis: resultados

	<i>StdSize / LargeSize</i>	
<i>Comparação</i>	<i>Chi-quadrado</i>	<i>p-valor</i>
arq. comunicação	21,806 / 22,500	< 2,2e-16 / < 2,2e-16
linguagem	1,239,3 / 356,88	< 2,2e-16 / < 2,2e-16
fator combinado	24,624 / 22,988	< 2,2e-16 / < 2,2e-16

Ranking. Considerando que há uma diferença estatisticamente significativa entre as medianas, é possível classificar o desempenho dos pares. A Tabela 2.4 exibe a lista das implementações ordenadas da melhor para a pior, considerando as requisições *StdSize* e *LargeSize*. Essa ordenação identifica quais combinações de linguagem e arquitetura de comunicação oferecem o melhor desempenho, auxiliando na escolha das tecnologias mais eficientes para aplicações distribuídas.

Tabela 2.4 – *Rankings* das implementações

Posição	<i>StdSize</i>	<i>LargeSize</i>
#1	(Go, gRPC), 0,00071s	(Go, REST), 0,00447s
#2	(Java, gRPC), 0,00119s	(Java, REST), 0,00451s
#3	(Java, REST), 0,00321s	(Go, gRPC), 0,00768s
#4	(Go, REST), 0,00323s	(Java, gRPC), 0,00803s

Nesses resultados, observa-se uma influência significativa do tamanho da requisição, *StdSize* ou *LargeSize*, e da combinação entre linguagem e estilo arquitetural de comunicação sobre o desempenho das implementações. Os resultados indicam que (Go, gRPC) mostrou superioridade em requisições *StdSize*, enquanto (Go, REST) e (Java, REST) se destacaram – quase de forma equivalente – nas requisições *LargeSize*. Essa distinção é particularmente relevante no projeto de sistemas distribuídos, onde a escolha do estilo arquitetural de comunicação pode afetar significativamente a eficiência operacional. Por um lado, a arquitetura de comunicação gRPC, com sua forma de comunicação mais leve e baseado em RPC, provou ser mais eficiente em requisições usuais, possivelmente devido à eficiência na compactação de dados e na redução da latência e da sobrecarga de comunicação. Por outro lado, a arquitetura de comunicação REST mostrou-se mais adaptada para requisições de grande volume, provavelmente devido à flexibilidade do REST em lidar com grandes volumes de dados sem estrutura rígida.

2.3.7 Influência dos Fatores (QP#2)

Esta seção provê respostas à questão de pesquisa #2 por meio da condução de um projeto estatístico nas cinco repetições dos dados coletados conforme descrito na Seção 2.3.5.

Para entender o impacto de cada fator no desempenho das aplicações distribuídas, utilizou-se o projeto fatorial $2^k r$ com 2 fatores e 5 repetições. Esse projeto experimental permite a análise sistemática dos efeitos principais e das interações entre os fatores (JAIN, 1991). O projeto $2^k r$ é uma extensão do projeto fatorial 2^k , incluindo repetições para estimar o erro experimental e aumentar a precisão dos resultados.

Os fatores escolhidos para o experimento foram a linguagem de programação e o estilo arquitetural de comunicação. A linguagem de programação (A) variou entre Go (-1) e Java (+1), enquanto o estilo arquitetural de comunicação (B) variou entre gRPC (-1) e REST (+1). Esses fatores são fundamentais para aplicações distribuídas modernas, onde a escolha da tecnologia pode impactar significativamente o desempenho.

Tabela 2.5 reporta, respectivamente, as tabelas de sinais utilizadas para o projeto fatorial $2^2 r$ para requisições *StdSize* e *LargeSize* incluindo os totais por fator e os totais divididos por 4, também conhecido como efeito do fator. Foram combinadas as duas tabelas de sinais pois apenas as duas última linhas se diferem. Por exemplo, pode-se dizer que o efeito do fator linguagem de programação para requisições *StdSize* foi de 0,00017 e para requisições *LargeSize* foi de 0,00019.

Com os efeitos de cada fator, são calculadas as seguintes sete somas de quadrados para obter de fato as influências de cada fator: (i) SSY (*Soma dos Quadrados Total dos Dados*) que representa a totalidade da variação observada nos dados, somando os quadrados de cada valor de resposta coletado; (ii) SSE (*Erro da Soma dos Quadrados*) que corresponde à variação em Y que não é explicada pelos fatores no modelo e é calculada subtraindo a contribuição dos fatores e suas intera-

Tabela 2.5 – Projeto Fatorial 2^2r - *StdSize*

I	Ling.	Arq. com.	Interação	Impl.
1	-1	-1	1	(Go, gRPC)
1	1	-1	-1	(Java, gRPC)
1	-1	1	-1	(Go, REST)
1	1	1	1	(Java, REST)
0,00973	0,00070	0,00526	-0,00082	$Total_{Std}$
0,00243	0,00017	0,00131	-0,00020	$Total_{Std}/4$
0,02486	0,00079	-0,00599	-0,00049	$Total_{Large}$
0,00621	0,00019	-0,00149	-0,00012	$Total_{Large}/4$

ções do SSY; (iii) SS0 (*Soma dos Quadrados do Total*) que é a soma associada ao efeito total, que engloba todos os efeitos sistemáticos observados; (iv) SST (*Soma dos Quadrados dos Tratamentos*) é a variação que é explicada apenas pelos tratamentos, calculada subtraindo SS0 de SSY; (v) SSL (*Soma dos Quadrados da Linguagem*) que quantifica o quanto da variação total em Y é explicada pelas diferenças entre as linguagens de programação utilizadas; (vi) SSAC (*Soma dos Quadrados da Arquitetura de Comunicação*) que avalia o impacto do estilo arquitetural de comunicação sobre a variação nos dados; e (vii) SSLAC (*Soma dos Quadrados da Interação Linguagem e Arquitetura de Comunicação*) que mede a interação entre os fatores Linguagem e Arquitetura de Comunicação, indicando se o efeito de uma variável sobre o desempenho depende da outra.

Dado que E_T é o Efeito Total, E_L é o Efeito Linguagem de Programação, E_{AC} é o Efeito Arquitetura de Comunicação, e E_I : Efeito Interação, as Equações 2 a 8 transcrevem as fórmulas para cada soma de quadrados incluindo os valores resultantes no formato $v_{StdSize}/v_{LargeSize}$, onde $v_{StdSize}$ é o resultado para o projeto fatorial de requisições *StdSize* e $v_{LargeSize}$ de requisições *LargeSize*.

$$SSY = \sum(Y_{ij}^2) = 0,00015/0,00081 \quad (2.2)$$

$$SSE = SSY - 2^k r (E_T^2 + E_L^2 + E_{AC}^2 + E_I^2) = 8,84833/5,74272 \quad (2.3)$$

$$SS0 = 2^k r E_T^2 = 0,00011/0,00077 \quad (2.4)$$

$$SST = SSY - SS0 = 3,70512/4,65600 \quad (2.5)$$

$$SSL = 2^k r E_L^2 = 6,20191/7,90081 \quad (2.6)$$

$$SSAC = 2^k r E_{AC}^2 = 3,47041/4,48847 \quad (2.7)$$

$$SSLAC = 2^k r E_I^2 = 8,42123/3,10969 \quad (2.8)$$

Por fim, as influências de cada fator no resultado final são calculadas como porcentagens das somas de quadrados sobre a soma total dos quadrados. A Tabela 2.6 reporta, portanto, as influências dos fatores para as requisições *StdSize* e *LargeSize*.

Tabela 2.6 – Influência dos Fatores no Desempenho

Fator	<i>StdSize</i>	<i>LargeSize</i>
Linguagem de programação	1,67%	1,69%
Arquitetura de comunicação	93,66%	96,40%
Interação	2,27%	0,66%
Erro	2,38%	1,23%

A aplicação do projeto fatorial $2^k r$ mostrou que o estilo arquitetural de comunicação é o principal fator que influencia o desempenho de aplicações distribuídas, sendo responsável por aproximadamente 94% do impacto em requisições de tamanho padrão (*StdSize*) e superando 96% em requisições de grande volume (*LargeSize*).

A influência da linguagem de programação, embora presente, é menos pronunciada comparativamente ao estilo arquitetural de comunicação. Esses resultados enfatizam a importância de selecionar o estilo arquitetural de comunicação adequado para otimizar o desempenho de sistemas distribuídos, oferecendo um suporte claro para decisões técnicas no desenvolvimento de software distribuído.

A influência da interação foi até superior à influência da linguagem de programação para requisições *StdSize*, porém praticamente nula (0,66%) para requisições *LargeSize*, o que indica que a interação é contextual e dependente do tamanho da requisição, sendo mais relevante em cenários com cargas menores e menos impactante em situações com volumes de dados maiores.

Por fim, a influência do erro é relativamente pequena, indicando que o modelo experimental utilizado é robusto e confiável.

2.3.8 Ameaças à validade

Pelo menos três ameaças à validade podem ser mencionadas:

Hardware e software. Otimizações das linguagens de programação, de processadores, uso de memória e interferência dos serviços do Ubuntu podem ter afetado as mensurações de tempo. No entanto, pode-se alegar que todas as decisões de projeto do experimento foram tomadas antes de qualquer mensuração e afeta as linguagens de programação e os estilos arquiteturais de comunicação da mesma forma.

Tamanho das requisições. Embora os tamanhos *StdSize* e *LargeSize* tenham sido projetados para representar, respectivamente, cargas médias e substancialmente maiores que a média, a definição de “média” pode não capturar adequadamente a variabilidade dos tamanhos de requisição encontrados em aplicações do mundo real. Trabalhos futuros na linha de uso de mais métricas são propostos para ajudar a refletir a distribuição real de tamanhos de requisição em aplicações distribuídas, onde requisições podem variar significativamente em tamanho e frequência.

Aplicação. No escopo deste estudo, a aplicação alvo foi projetada propositalmente para isolar – ao máximo – o experimento de fatores externos. De forma complementar, experimentos com aplicações que demandem aspectos particulares das

linguagens de programação, como concorrência e paralelismo, são vislumbrados como trabalhos futuros.

Essas ameaças destacam a necessidade de cautela ao interpretar os resultados e ao considerar a aplicabilidade das conclusões para outros contextos ou configurações. É essencial reconhecer que, enquanto os resultados fornecem *insights* valiosos sobre o desempenho relativo das tecnologias estudadas sob condições específicas, a generalização desses resultados para outros cenários deve ser feita com uma compreensão clara das limitações e contextos em que os testes foram realizados.

2.3.9 Replicabilidade

Os aplicativos utilizados e os dados coletados durante o estudo estão publicamente disponíveis em:

[<https://github.com/PqES/2024_sb1p>](https://github.com/PqES/2024_sb1p)

2.4 Trabalhos Relacionados

Esta seção descreve trabalhos relacionados ao estudo empírico conduzido neste artigo.

Śliwa e Pańczyk (ŚLIWA; PAŃCZYK, 2021) avaliam o desempenho das arquiteturas de comunicação REST, GraphQL e gRPC, destacando-se o REST pela quantidade de transações por segundo e o gRPC pela eficiência na transmissão de dados. O estudo proporciona um pano de fundo relevante para a presente pesquisa, que também examina o desempenho de diferentes arquiteturas de comunicação, mas se preocupa com o fator das linguagens Java e Go, além de utilizar métodos estatísticos robustos para avaliar o impacto dessas arquiteturas em diversos tamanhos de requisição. Contrariamente, a pesquisa inclui a análise de GraphQL e não emprega análises estatísticas avançadas como o teste de Wilcoxon *Rank-Sum*

ou Kruskal-Wallis utilizados neste trabalho. Portanto, enquanto ambos os estudos contribuem para a compreensão do desempenho de diferentes tecnologias de comunicação, o presente trabalho distingue-se pela sua abordagem metodológica e foco analítico específico.

Bora e Bezboruah (BORA; BEZBORUAH, 2015) publicaram dois serviços web distintos, um utilizando a arquitetura SOAP e outro com arquitetura REST, ambos implementados em Java. Eles realizaram uma avaliação comparativa desses serviços, empregando o *Mercury Load Runner* para testes de carga e estresse. A análise estatística dos resultados revelou que a arquitetura REST mostrou-se mais eficiente e escalável em comparação com SOAP. Esse resultado é semelhante ao encontrado no presente trabalho, onde a arquitetura REST também mostrou maior eficiência em requisições de grande volume, embora o presente estudo usa uma abordagem estatística mais rigorosa com testes específicos para medir diferenças de desempenho.

Abhinav et al. (ABHINAV et al., 2020) analisa a concorrência entre Go e Java por meio de experimentos com algoritmos de multiplicação de matrizes e *PageRank*. Os autores identificam que Go apresenta melhor desempenho em menor número de cálculos, enquanto Java supera à medida que o volume de cálculos aumenta. Esse padrão se inverte quando considerado a criação e gerenciamento de *threads*, com Java mostrando vantagem em computações menores e Go melhorando em cenários mais exigentes. Ambas as linguagens possuem características únicas de concorrência, destacando-se em diferentes aplicações. No contexto do presente trabalho, não existem cálculos numerosos sendo realizados nas implementações alvo, isso pode explicar a baixa influência da linguagem de programação nos resultados.

Marx et al. (MARX et al., 2018) analisam as características de desempenho do HTTP/2 em comparação ao HTTP/1.1. Os experimentos revelam que, apesar das melhorias propostas, o HTTP/2 apresenta limitações significativas em con-

dições de rede adversas, onde a conexão multiplexada pode se tornar um gargalo e os benefícios em termos de desempenho raramente superam os do HTTP/1.1, exceto na redução de sobrecarga. É relevante mencionar que a arquitetura de comunicação gRPC, utiliza o HTTP/2 como base, o que influencia diretamente o seu desempenho em comparação ao REST, que tradicionalmente utiliza o HTTP/1.1.

Kamiński et al. (KAMIŃSKI et al., 2023) conduzem uma análise comparativa das arquiteturas de comunicação de Internet mais usados, incluindo REST, *WebSocket*, gRPC, GraphQL e SOAP. Utilizando servidores web implementados em Python, a equipe mediu o tempo e a sobrecarga de transferência de dados em operações específicas. Concluiu-se que o gRPC foi o método mais eficiente e confiável, enquanto o GraphQL mostrou-se o mais lento e problemático em termos de uso em servidores web. Esse estudo utilizou cargas pequenas para os comparativos e seus resultados em relação às arquiteturas de comunicação corroboram com os obtidos no presente trabalho com cargas semelhantes.

Berg e Redi (BERG; REDI, 2023) conduziram um estudo comparativo sobre o desempenho de *throughput* de requisições entre chamadas de API convencionais baseadas em REST e gRPC. O estudo foi realizado com uma metodologia bem definida, desenvolvida pelos próprios autores, sem o uso de métodos estatísticos tradicionais. Os servidores foram implementados em três linguagens de programação (Java, Python e Rust) e os testes de *benchmark* foram conduzidos em uma rede local para avaliar o desempenho de requisições em diferentes tamanhos de carga. Os resultados indicaram que, embora o REST inicie com um desempenho melhor para cargas menores, o gRPC supera o REST à medida que o tamanho da carga útil aumenta, oferecendo um *throughput* superior em mensagens de maior tamanho. Embora possa parecer que o resultado é o oposto do encontrado neste estudo, os estudos se complementam e apresentam comportamentos opostos em diferentes métricas (*throughput* vs. tempo de execução).

2.5 Conclusão

A motivação para este estudo surgiu da necessidade crescente de desenvolver APIs robustas e eficientes que possam atender às exigentes demandas de desempenho e escalabilidade de sistemas de software atuais. Diante disso, foi conduzido um estudo empírico que emprega procedimentos estatísticos rigorosos para quantificar as melhorias oferecidas por uma linguagem de programação e um estilo arquitetural de comunicação considerados promissores dentro de um contexto de sistemas distribuídos, abordando especificamente Go e Java e gRPC e REST.

A pesquisa foi impulsionada por duas principais questões de pesquisa: (i) qual a combinação mais eficiente entre as linguagens de programação Go e Java e os estilos arquiteturais de comunicação gRPC e REST e (ii) qual a influência da linguagem de programação e do estilo arquitetural de comunicação na eficiência.

A questão de pesquisa #1 evidenciou que Go apresenta desempenho superior tanto em requisições *StdSize* quanto *LargeSize*, comparado ao Java. Esse resultado sugere que a seleção de Go em combinação com gRPC, pode oferecer vantagens substanciais em termos de eficiência, especialmente em ambientes que requerem comunicações frequentes e de alto desempenho.

A questão de pesquisa #2 conclui que – de fato – a influência da arquitetura de comunicação é de 94% no desempenho. Isso indica que, ao substituir REST por gRPC, existem cenários em que fábricas de software podem ter uma melhoria de desempenho em seus sistemas sem necessitar de uma reescrita de código em uma linguagem de programação diferente.

A diferença de desempenho entre REST e gRPC nos diferentes tamanhos de requisição pode ser atribuída a diversos fatores inerentes às duas arquiteturas de comunicação. O gRPC, por ser baseado no protocolo HTTP/2 e utilizar a serialização Protobuf, é mais eficiente em termos de latência e consumo de recursos para transmissões de dados menores devido à sua capacidade de multiplexação de múl-

tiplas chamadas em uma única conexão. Já o REST, com sua simplicidade e ampla adoção, tende a ser mais eficiente para requisições maiores, possivelmente devido à sua capacidade de lidar com cargas de dados mais pesadas e menos dependência de conexões simultâneas.

Trabalhos futuros incluem: (i) avaliar a eficiência com requisições de diversos tamanhos a fim de encontrar pontos de transição onde uma arquitetura de comunicação pode superar outra; (ii) avaliar a eficiência com diferentes frequências de solicitações, a fim de obter os limiares de desempenho de REST e gRPC em situações de estresse; (iii) avaliar em diferentes configurações de infraestrutura como rede, CPU, memória, etc.; e (iv) avaliar o desempenho com algoritmos específicos no servidor, a fim de medir o impacto de funcionalidades específicas das linguagens, como paralelismo e gerenciamento de memória.

3 CONCLUSÃO

A motivação para este estudo surgiu da necessidade crescente de desenvolver APIs robustas e eficientes que possam atender às exigentes demandas de desempenho e escalabilidade de sistemas de software atuais. Diante disso, foi conduzido um estudo empírico que emprega procedimentos estatísticos rigorosos para quantificar as melhorias oferecidas por uma linguagem de programação e um estilo arquitetural de comunicação considerados promissores dentro de um contexto de sistemas distribuídos, abordando especificamente Go e Java e gRPC e REST.

A pesquisa foi impulsionada por duas principais questões de pesquisa: (i) qual a combinação mais eficiente entre as linguagens de programação Go e Java e os estilos arquiteturais de comunicação gRPC e REST e (ii) qual a influência da linguagem de programação e do estilo arquitetural de comunicação na eficiência.

A questão de pesquisa #1 evidenciou que Go apresenta desempenho superior tanto em requisições *StdSize* quanto *LargeSize*, comparado ao Java. Esse resultado sugere que a seleção de Go em combinação com gRPC, pode oferecer vantagens substanciais em termos de eficiência, especialmente em ambientes que requerem comunicações frequentes e de alto desempenho.

A questão de pesquisa #2 conclui que – de fato – a influência da arquitetura de comunicação é de 94% no desempenho. Isso indica que, ao substituir REST por gRPC, existem cenários em que fábricas de software podem ter uma melhoria de desempenho em seus sistemas sem necessitar de uma reescrita de código em uma linguagem de programação diferente.

Trabalhos futuros incluem: (i) avaliar a eficiência com requisições de diversos tamanhos a fim de encontrar pontos de transição onde uma arquitetura de comunicação pode superar outra; (ii) avaliar a eficiência com diferentes frequências de solicitações, a fim de obter os limiares de desempenho de REST e gRPC em situações de estresse; (iii) avaliar em diferentes configurações de infraestrutura

como rede, CPU, memória, etc.; e (iv) avaliar o desempenho com algoritmos específicos no servidor, a fim de medir o impacto de funcionalidades específicas das linguagens, como paralelismo e gerenciamento de memória.

REFERÊNCIAS

ABHINAV, P. Y. et al. Concurrency analysis of go and java. In: **5th International Conference on Computing, Communication and Security (ICCCS)**. [S.l.: s.n.], 2020. p. 1–6.

ASRATIAN, R. Safe internet tunnel with access control features to web-services based on ssl/tls protocol. In: **1st International Multi-Conference on Advanced Computing and Communication Technologies (MLSD)**. [S.l.: s.n.], 2023.

BELSHE, M.; PEON, R.; THOMSON, M. **Hypertext Transfer Protocol Version 2 (HTTP/2)**. 2015. Internet Requests for Comments. Disponível em: <<https://tools.ietf.org/html/rfc7540>>.

BERG, J.; REDI, D. M. **Benchmarking the Request Throughput of Conventional API Calls and gRPC: A Comparative Study of REST and gRPC**. Dissertação (Bachelor's Thesis) — School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, Stockholm, Sweden, 2023. Disponível em: <<https://github.com/jonaro00/rest-vs-grpc>>.

BIRRELL, A.; NELSON, B. Implementing remote procedure calls. **ACM Transactions on Computer Systems (TOCS)**, v. 2, n. 1, p. 39–59, 1984.

BOLANOWSKI, M. et al. Efficiency of rest and grpc realizing communication tasks in microservice-based ecosystems. In: _____. Amsterdam, Netherlands: IOS Press, 2022. p. 97–108. ISBN 9781643683164.

BORA, A.; BEZBORUAH, T. A comparative investigation on implementation of restful versus soap based web services. **International Journal of Database Theory and Application**, v. 8, n. 3, p. 297–312, 2015.

CHAVAN, P.; KADAM, A.; MISTRY, K. Optimizing software development: A comparative study of Python and R for enhanced efficiency and quality. **International Journal for Research in Applied Science and Engineering Technology**, v. 11, n. 1, p. 1969–1974, 2023.

COX, R. et al. The Go programming language and environment. **Communications of the ACM**, v. 65, n. 5, p. 70–77, 2022.

CROCKFORD, D. **The application/json Media Type for JavaScript Object Notation (JSON)**. RFC Editor, 2006. RFC 4627. (Request for Comments, 4627). Disponível em: <<https://www.rfc-editor.org/info/rfc4627>>.

DATTA, S.; SATTEN, G. A. Rank-sum tests for clustered data. **Journal of the American Statistical Association**, v. 100, n. 471, p. 908–915, 2005.

DONOVAN, A. A. A.; KERNIGHAN, B. W. **The Go Programming Language**. Boston, MA: Addison-Wesley Professional, 2012.

FIELDING, R. et al. **Hypertext Transfer Protocol – HTTP/1.1**. 1999. Internet Requests for Comments. Disponível em: <<https://tools.ietf.org/html/rfc2616>>.

FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. Tese (Doutorado) — University of California, Irvine, 2000.

GOOGLE. **gRPC: A high performance, open source universal RPC framework**. 2015. <<https://grpc.io>>.

HABIBZADEH, F. Data distribution: Normal or abnormal? **Journal of Korean Medical Science**, v. 39, n. e35, p. 1–8, 2024.

HOLOVNIA, O. S. et al. Interactive surveys during online lectures for it students. **Contemporary Educational Technology**, v. 13, n. 1, p. ep56, 2023.

JAIN, R. **The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling**. New York, NY, USA: Wiley-Interscience, 1991.

KAMIŃSKI, L. et al. Comparative review of selected internet communication protocols. **Foundations of Computing and Decision Sciences**, v. 48, n. 1, p. 39–56, 2023.

KUMAR, K. et al. Analysis of api architecture: A detailed report. In: **12th International Conference on Communication Systems and Network Technologies (CSNT)**. [S.l.: s.n.], 2023. p. 880–884.

LEE, Y.; LIU, Y. Using refactoring to migrate rest applications to grpc. In: **ACM Southeast Conference**. [S.l.: s.n.], 2022. p. 219–223.

MARX, R. et al. Web performance characteristics of http/2 and comparison to http/1.1. In: **Web Information Systems and Technologies**. [S.l.: s.n.], 2018. p. 87–114.

MIOT, H. A. Avaliação da normalidade dos dados em estudos clínicos e experimentais. **Jornal Vascular Brasileiro**, v. 16, n. 2, p. 88–91, 2017.

POKROPP, F. A note on conditional wilcoxon tests with natural and mid-ranks. **Biometrical Journal**, v. 34, n. 8, p. 859–863, 1992.

R Core Team. **R: A Language and Environment for Statistical Computing**. Vienna, Austria, 2019. Disponível em: <<https://www.R-project.org/>>.

SINGHAL, S.; NGUYEN, B. Q. The java factor. **ACM**, v. 41, n. 6, p. 34–37, 1998.

The Chromium Projects. **SPDY Protocol**. 2012. <<https://www.chromium.org/spdy/spdy-whitepaper>>. Acessado em 13 de fevereiro de 2024.

TIOBE Software. **TIOBE Index for May 2024**. 2024. <<https://www.tiobe.com/tiobe-index/>>.

ŚLIWA, M.; PAŃCZYK, B. Performance comparison of programming interfaces on the example of rest api, graphql and grpc. **Journal of Computer Sciences Institute**, v. 21, n. 21, p. 356–361, 2021.